

Perceptron-Based Classification and Regression

CS601T: Deep Learning – Programming Assignment I

Group 12

Submitted by

Dattaraj Balkrishna Saudagar (Roll No.: CS24BT037)

Onkar Avinash Mulje (Roll No.: CS24BT059)

Nimitt Jain (Roll No.: CS24BT048)



**Department of Computer Science and Engineering
Indian Institute of Technology Dharwad**

August 2026

Contents

Contents	2
Abstract	3
1. Introduction	4
2. Methodology	5
2.1 Perceptron Model	5
2.2 Gradient-Descent Learning Rule	5
2.3 One-Against-One Multi-Class Strategy	5
2.4 Evaluation Metrics (implemented from scratch)	6
3. Experimental Setup	7
3.1 Datasets	7
3.2 Train / Test Split	7
3.3 Hyperparameters	7
4. Results	8
4.1 Classification Results — Dataset 1 (Linearly Separable)	8
4.2 Classification Results — Dataset 2 (Non-Linearly Separable)	13
4.3 Regression Results — Dataset 1 (Univariate)	18
4.4 Regression Results — Dataset 2 (Bivariate)	20
5. Inferences and Observations	22
5.1 Classification	22
5.2 Regression	22
6. Conclusion	23

Abstract

This report presents the design, implementation, and evaluation of a single-layer perceptron for both classification and regression tasks, implemented entirely from scratch (without using any perceptron, neural network, or gradient-descent library). For classification, perceptrons with logistic and tan-hyperbolic activation functions were trained using a one-against-one strategy on a linearly separable (LS) and a non-linearly separable (NLS) three-class, two-dimensional dataset. For regression, a perceptron with a linear activation function was trained on a univariate and a bivariate dataset. All models were trained using the online gradient-descent learning rule. Performance is reported using confusion matrices, accuracy, class-wise and mean precision/recall/F-measure for classification, and RMSE / %RMSE for regression, all computed using custom implementations. Results show perfect separation on the linearly separable data and the expected performance degradation on the non-linearly separable and non-linear regression data, illustrating the fundamental limitation of a single linear decision boundary.

1. Introduction

The perceptron is the simplest artificial neuron: it computes a weighted sum of its inputs (the net input), passes this sum through a non-linear activation function, and produces a single scalar output. Despite its simplicity, the perceptron is the fundamental building block of larger neural networks, and understanding its learning rule and its limitations motivates the need for multi-layer architectures.

This assignment implements the perceptron learning algorithm from first principles and applies it to two kinds of tasks:

Classification: a 3-class, 2-dimensional problem is solved using a one-against-one ensemble of perceptrons, once with a logistic activation function and once with a tan-hyperbolic (tanh) activation function, on two datasets — one linearly separable (LS) and one non-linearly separable (NLS).

Regression: a perceptron with a linear activation function is trained to predict a continuous target from a univariate input and, separately, from a bivariate input.

In both cases the weights are updated using the online (per-sample) gradient-descent rule that minimizes the mean-squared error between the perceptron's output and the target, and every evaluation metric (confusion matrix, precision, recall, F-measure, RMSE, %RMSE) is implemented from scratch as required by the assignment.

2. Methodology

2.1 Perceptron Model

Each perceptron computes a net input as a weighted sum of the input features plus a bias, and applies an activation function f to obtain its output $o = f(\text{net})$:

$$\text{net} = \sum_{i=1}^d w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

Three activation functions are used in this assignment:

Logistic (sigmoid) — used for classification, output in (0, 1):

$$f(\text{net}) = \frac{1}{1 + e^{-\text{net}}}, \quad f'(\text{net}) = f(\text{net}) (1 - f(\text{net}))$$

Tan-hyperbolic (tanh) — used for classification, output in (-1, 1):

$$f(\text{net}) = \tanh(\text{net}), \quad f'(\text{net}) = 1 - f(\text{net})^2$$

Linear — used for regression, output unbounded:

$$f(\text{net}) = \text{net}, \quad f'(\text{net}) = 1$$

2.2 Gradient-Descent Learning Rule

Training minimizes the mean-squared error between target t and output o over the N training samples:

$$E = \frac{1}{2N} \sum_{k=1}^N (t_k - o_k)^2$$

Differentiating this error with respect to each weight gives the online (per-sample) gradient-descent update rule, applied after every training example with learning rate η :

$$w_i \leftarrow w_i + \eta (t - o) f'(\text{net}) x_i, \quad b \leftarrow b + \eta (t - o) f'(\text{net})$$

For the logistic and tanh activations the targets are encoded as {0,1} and {-1,+1} respectively; for the linear activation used in regression, $f'(\text{net}) = 1$ and the target is the real-valued output directly. Weights are initialized to small random values and the dataset order is reshuffled every epoch.

2.3 One-Against-One Multi-Class Strategy

Since a single perceptron only produces a binary decision, the 3-class classification problem is solved with three pairwise perceptrons: Class1-vs-Class2, Class1-vs-Class3, and Class2-vs-Class3, each trained only on the samples belonging to that pair of classes. At prediction time, all three pairwise classifiers vote for one of their two classes, and the class with the majority of votes is chosen as the final label; ties (possible when each classifier votes for a different class) are broken using the sum of the winning classifiers' confidence scores ($|\text{output} - \text{decision threshold}|$).

2.4 Evaluation Metrics (implemented from scratch)

For classification, a confusion matrix CM is built by counting true-label vs. predicted-label pairs on the test set. From it, per-class precision and recall are computed as:

$$\text{Precision}_j = \frac{CM_{jj}}{\sum_i CM_{ij}}$$

$$\text{Recall}_j = \frac{CM_{jj}}{\sum_j CM_{ij}}$$

and the per-class F-measure is their harmonic mean:

$$F_j = \frac{2 \text{Precision}_j \text{Recall}_j}{\text{Precision}_j + \text{Recall}_j}$$

Accuracy is the fraction of correctly classified test samples, and the mean precision, mean recall and mean F-measure are simple averages of the per-class values. For regression, the root-mean-squared error and percentage RMSE (RMSE normalized by the mean of the target values) are used:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{k=1}^N (t_k - o_k)^2}$$

$$\% \text{RMSE} = \frac{\text{RMSE}}{\bar{t}} \times 100$$

3. Experimental Setup

3.1 Datasets

Task	Dataset	Size	Description
Classification	LS (Dataset 1)	$3 \times 500 = 1500$	2-D, linearly separable
Classification	NLS (Dataset 2)	$3 \times 500 = 1500$	2-D, non-linearly separable
Regression	Univariate (Dataset 1)	1000	1-D input, 1-D output
Regression	Bivariate (Dataset 2)	10200	2-D input, 1-D output

3.2 Train / Test Split

For every dataset, each class (classification) or the whole dataset (regression) is split into 70% training data and 30% test data using a fixed random seed. The split is performed exactly once by a dedicated data-preparation step and the resulting indices are cached to disk; every subsequent training run (for any activation function or model configuration) loads this same cached split, so the train/test partition never changes when switching between models.

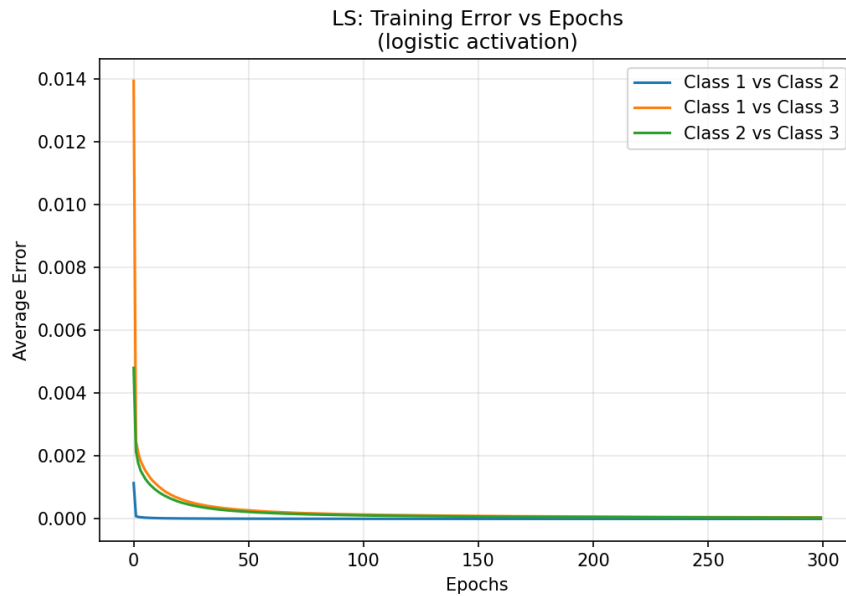
3.3 Hyperparameters

Experiment	Learning Rate	Max Epochs
Classification (LS, NLS)	0.05	300
Regression (Univariate)	0.01	200
Regression (Bivariate)	0.01	200

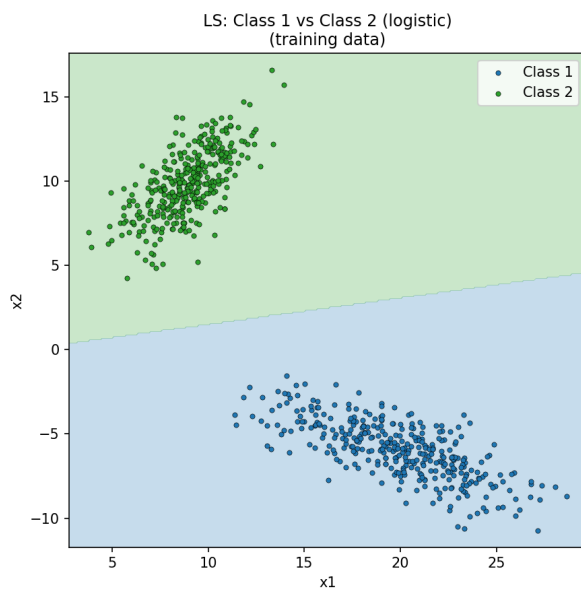
4. Results

4.1 Classification Results — Dataset 1 (Linearly Separable)

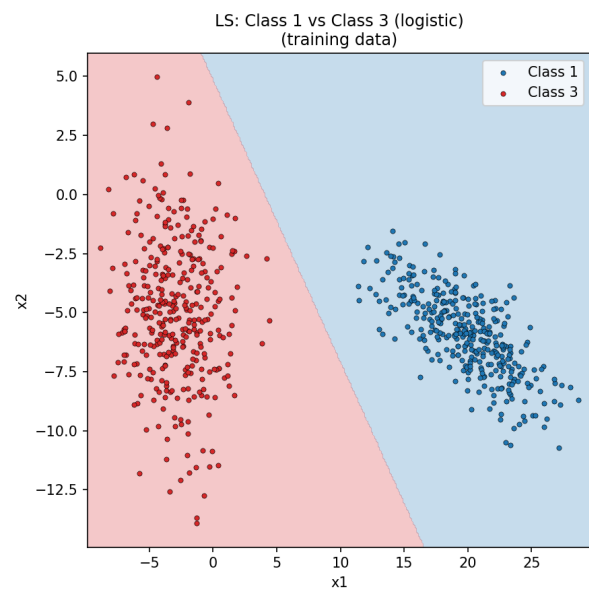
4.1.1 Logistic Activation



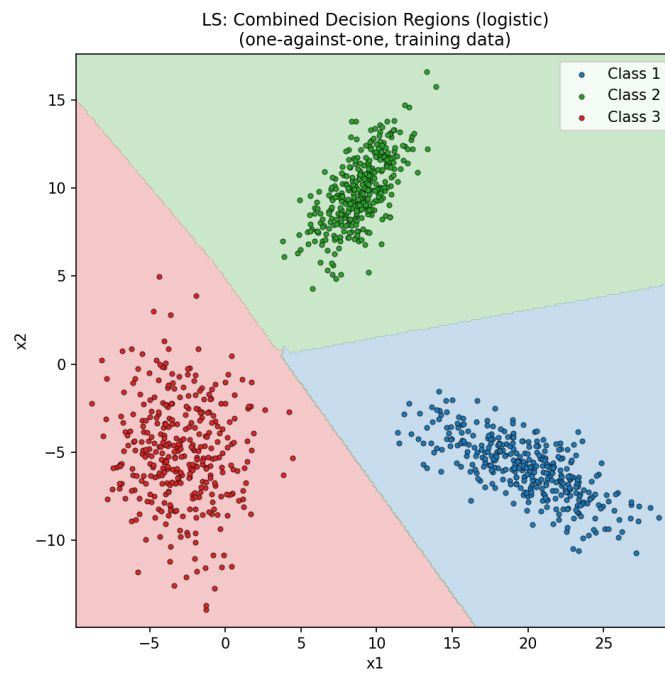
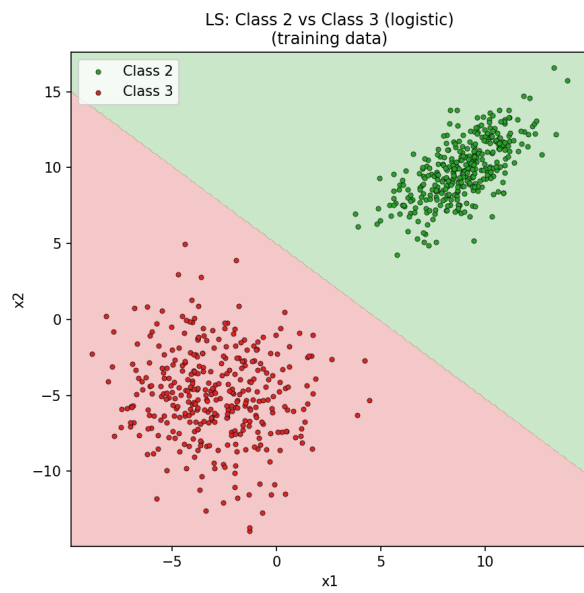
Average error vs. epochs for each one-against-one pair (Dataset 1 (Linearly Separable), Logistic Activation).



Class 1 vs Class 2



Class 1 vs Class 3



Combined decision regions after one-against-one voting (Dataset 1 (Linearly Separable), Logistic Activation), training data only.

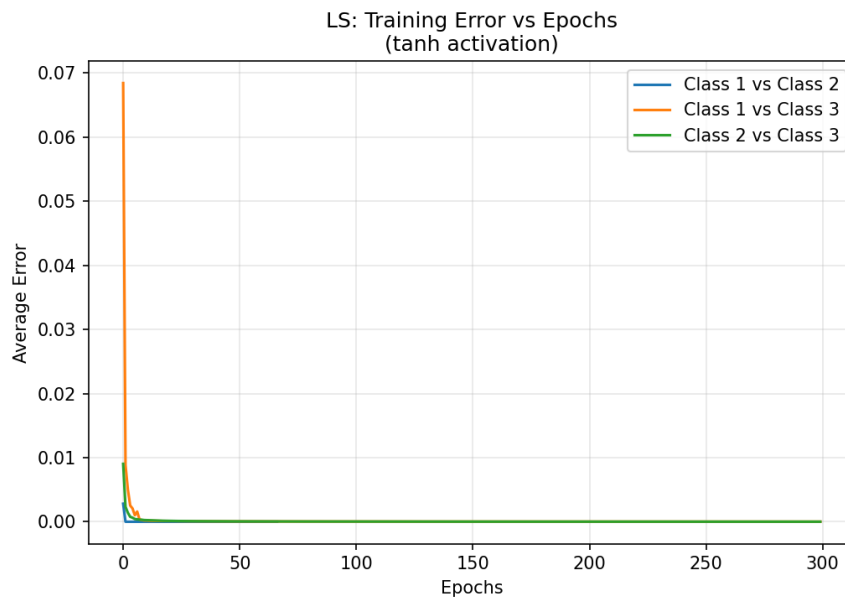
Confusion matrix (test data, Logistic Activation):

True \ Pred	Class 1	Class 2	Class 3
Class 1	150	0	0
Class 2	0	150	0
Class 3	0	0	150

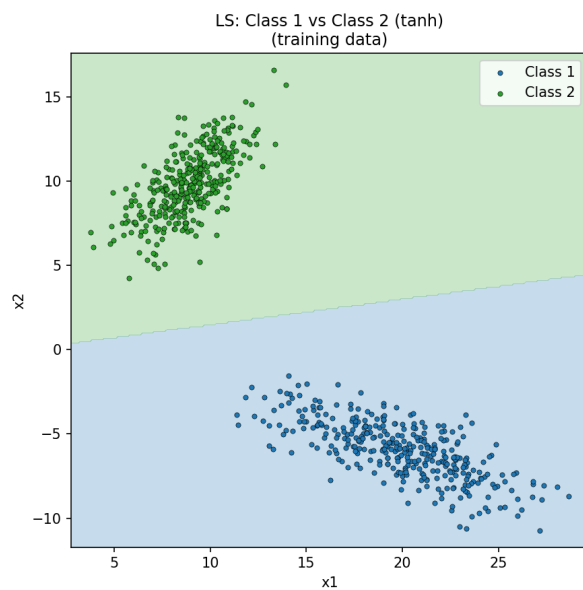
Test accuracy: 100.00%

Class	Precision	Recall	F-measure
Class 1	1.0000	1.0000	1.0000
Class 2	1.0000	1.0000	1.0000
Class 3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

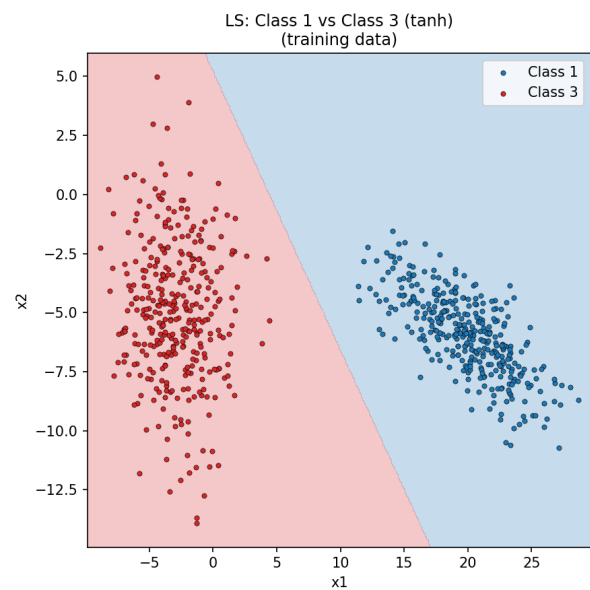
4.1.2 Tan-Hyperbolic Activation



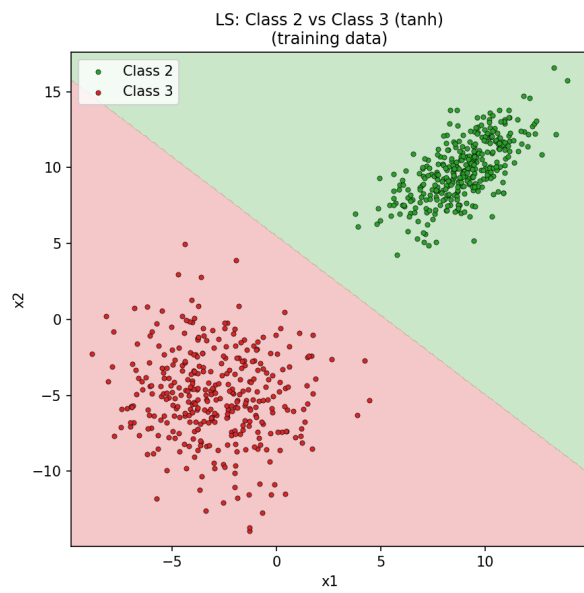
Average error vs. epochs for each one-against-one pair (Dataset 1 (Linearly Separable), Tan-Hyperbolic Activation).



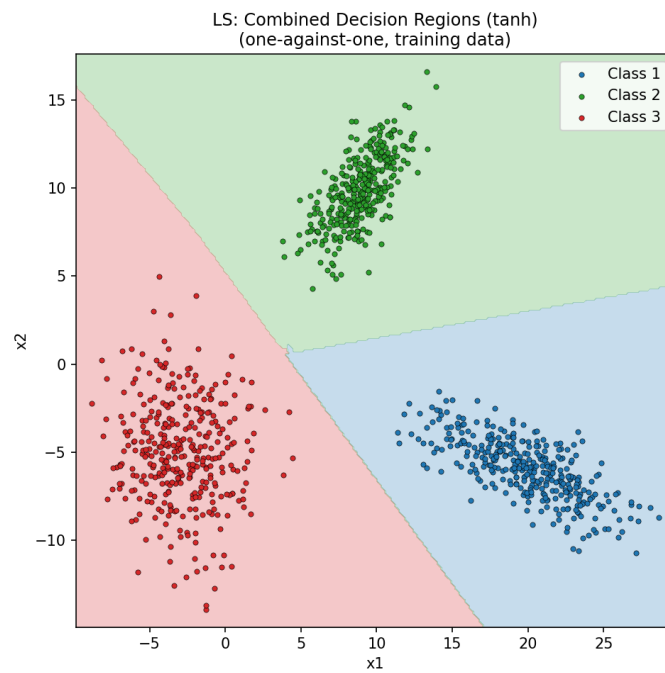
Class 1 vs Class 2



Class 1 vs Class 3



Class 2 vs Class 3



Combined decision regions after one-against-one voting (Dataset 1 (Linearly Separable), Tan-Hyperbolic Activation), training data only.

Confusion matrix (test data, Tan-Hyperbolic Activation):

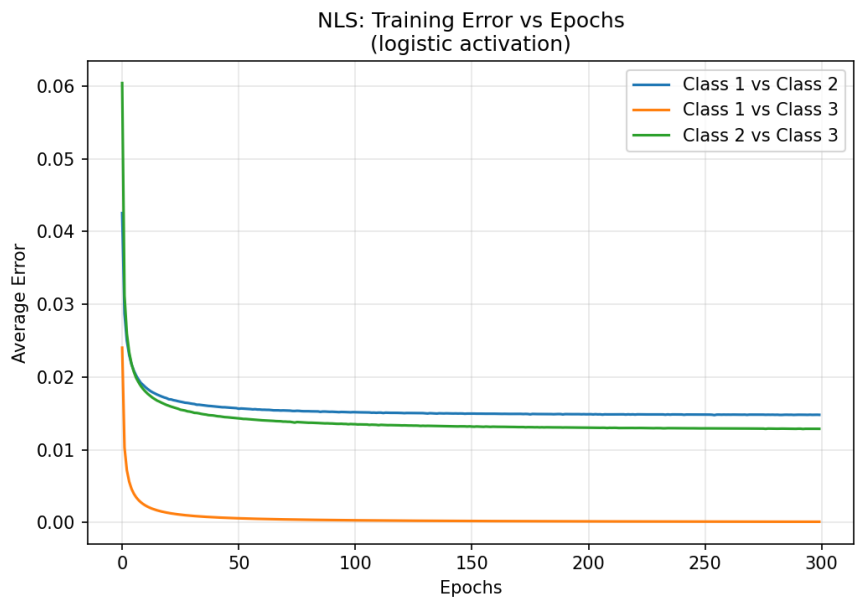
True \ Pred	Class 1	Class 2	Class 3
Class 1	150	0	0
Class 2	0	150	0
Class 3	0	0	150

Test accuracy: 100.00%

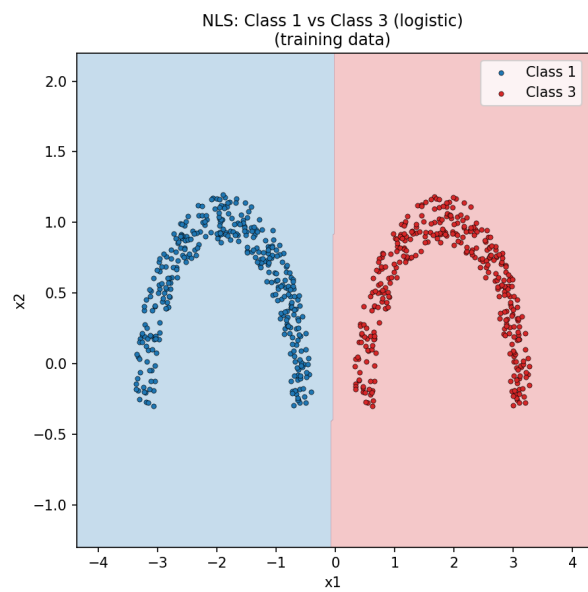
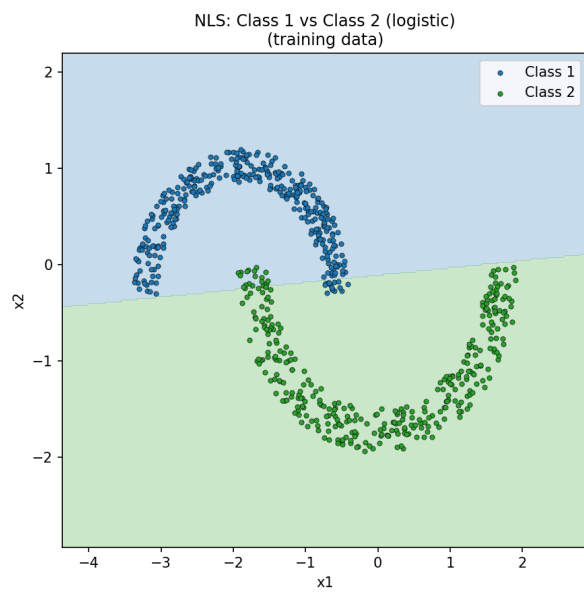
Class	Precision	Recall	F-measure
Class 1	1.0000	1.0000	1.0000
Class 2	1.0000	1.0000	1.0000
Class 3	1.0000	1.0000	1.0000
Mean	1.0000	1.0000	1.0000

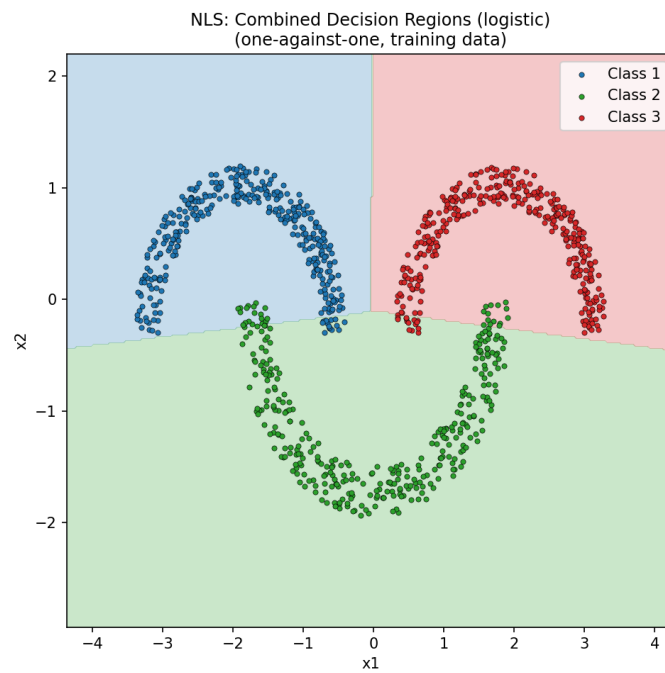
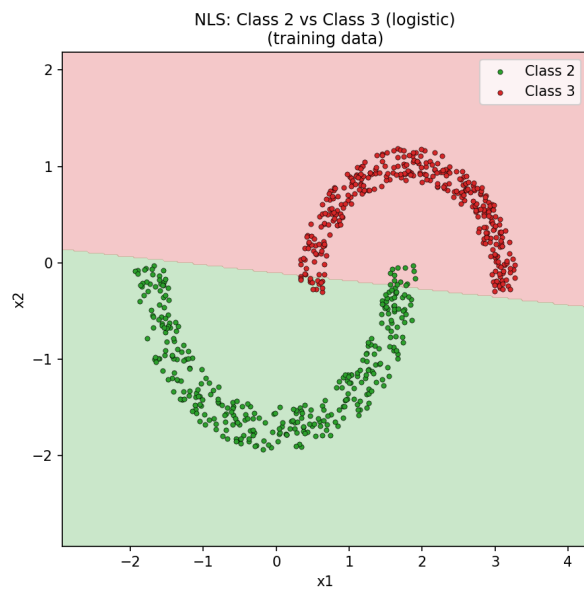
4.2 Classification Results — Dataset 2 (Non-Linearly Separable)

4.2.1 Logistic Activation



Average error vs. epochs for each one-against-one pair (Dataset 2 (Non-Linearly Separable), Logistic Activation).





Combined decision regions after one-against-one voting (Dataset 2 (Non-Linearly Separable), Logistic Activation),
training data only.

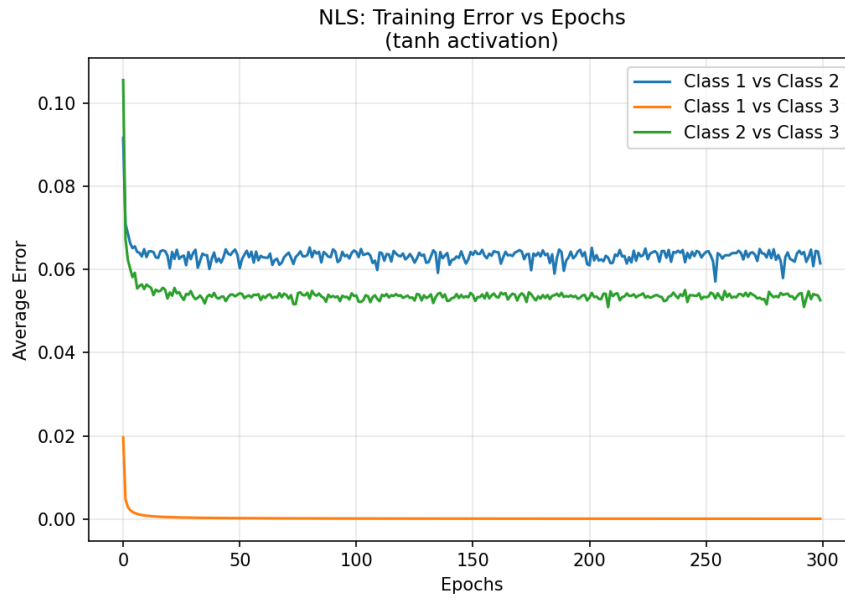
Confusion matrix (test data, Logistic Activation):

True \ Pred	Class 1	Class 2	Class 3
Class 1	142	8	0
Class 2	10	130	10
Class 3	0	4	146

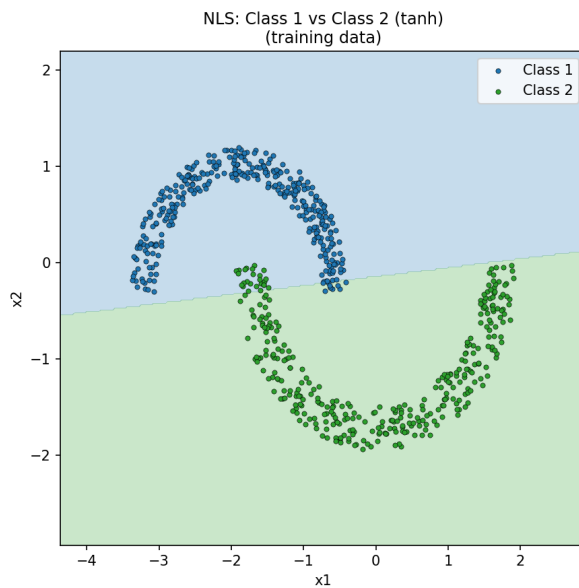
Test accuracy: 92.89%

Class	Precision	Recall	F-measure
Class 1	0.9342	0.9467	0.9404
Class 2	0.9155	0.8667	0.8904
Class 3	0.9359	0.9733	0.9542
Mean	0.9285	0.9289	0.9284

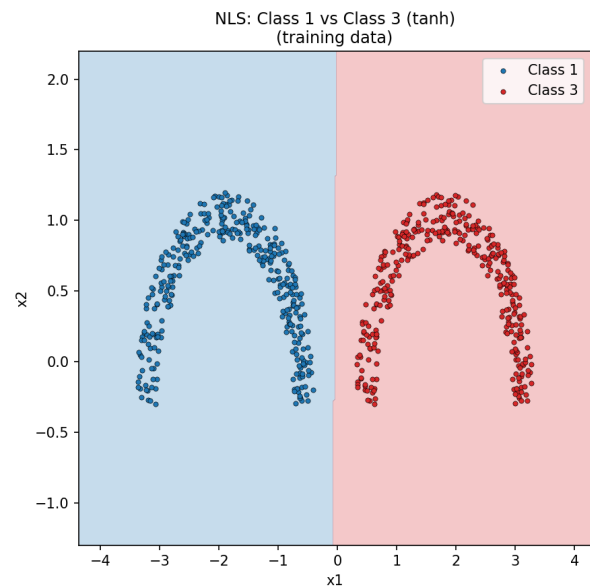
4.2.2 Tan-Hyperbolic Activation



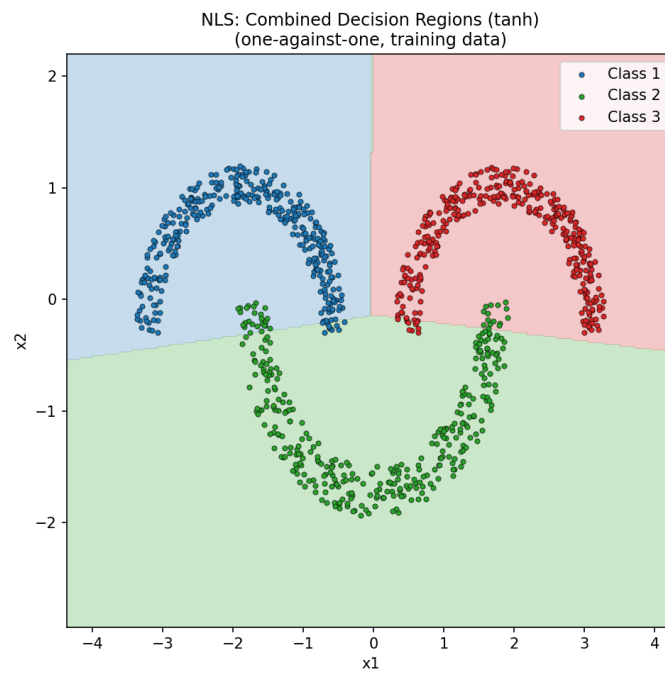
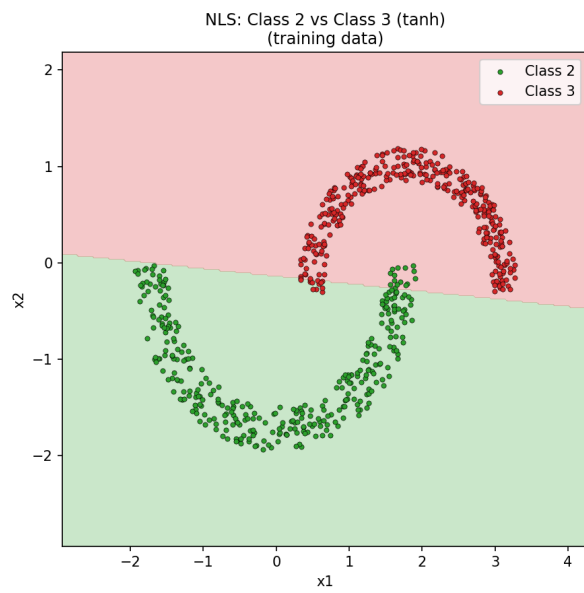
Average error vs. epochs for each one-against-one pair (Dataset 2 (Non-Linearly Separable), Tan-Hyperbolic Activation).



Class 1 vs Class 2



Class 1 vs Class 3



Combined decision regions after one-against-one voting (Dataset 2 (Non-Linearly Separable), Tan-Hyperbolic Activation), training data only.

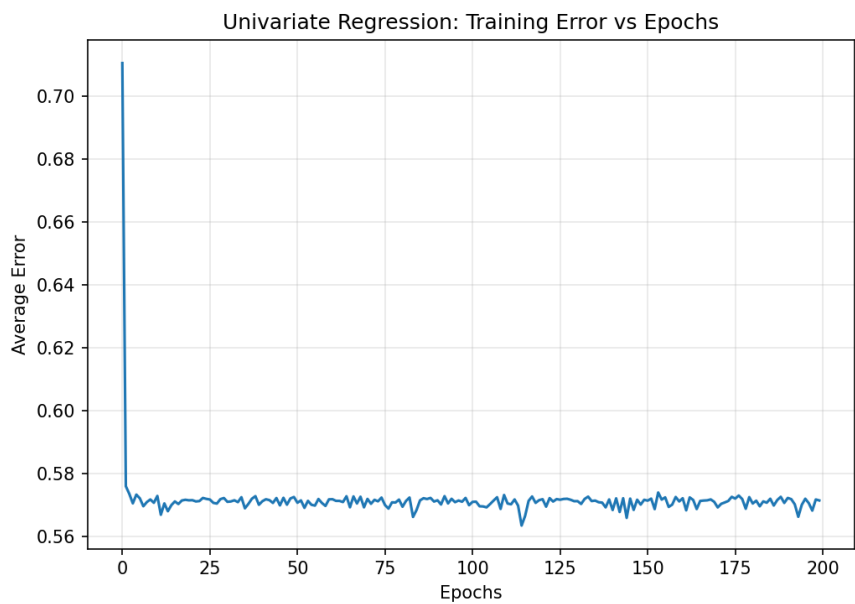
Confusion matrix (test data, Tan-Hyperbolic Activation):

True \ Pred	Class 1	Class 2	Class 3
Class 1	143	7	0
Class 2	12	128	10
Class 3	0	3	147

Test accuracy: 92.89%

Class	Precision	Recall	F-measure
Class 1	0.9226	0.9533	0.9377
Class 2	0.9275	0.8533	0.8889
Class 3	0.9363	0.9800	0.9577
Mean	0.9288	0.9289	0.9281

4.3 Regression Results — Dataset 1 (Univariate)

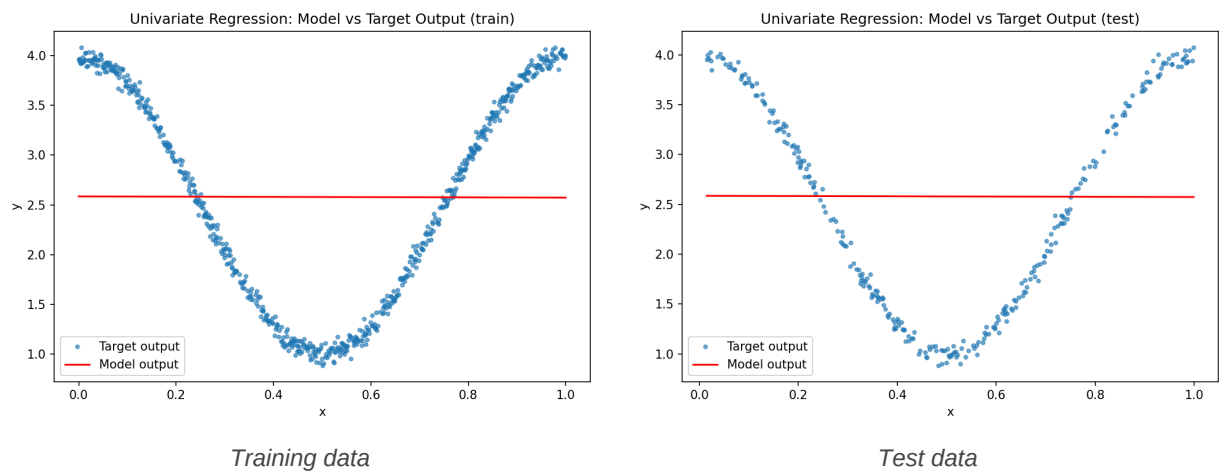


Average error vs. epochs (Dataset 1 (Univariate)).

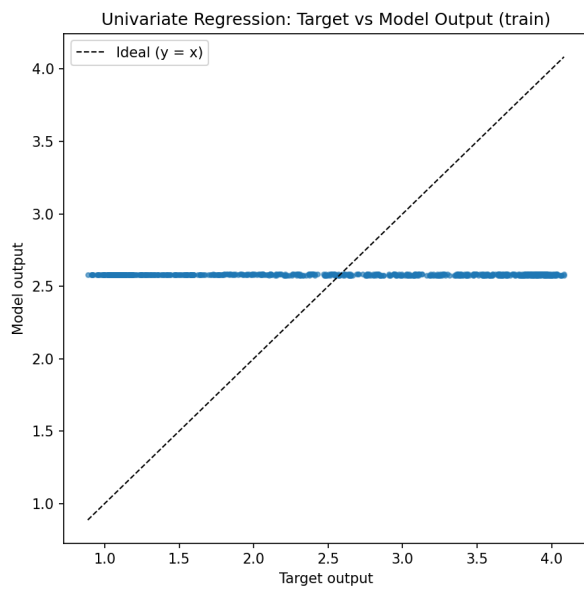
RMSE and %RMSE:

Split	RMSE	%RMSE
Training	1.0651	41.98%
Test	1.0607	43.88%

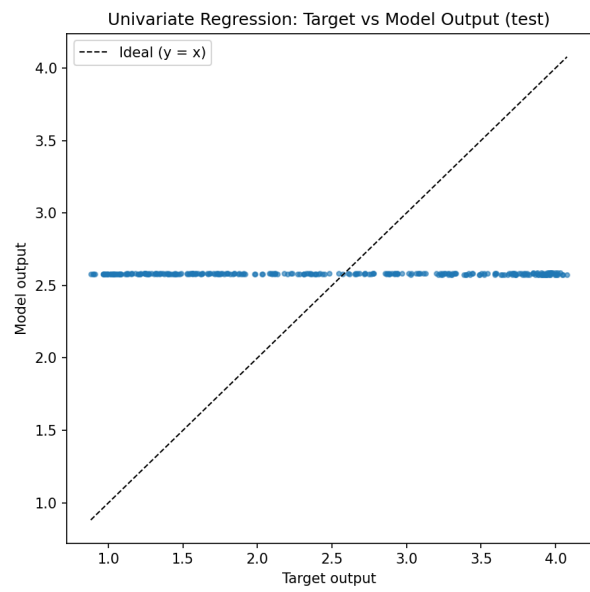
Model output superimposed on target output (x-axis: input x, y-axis: target / model output):



Scatter plot: target output (x-axis) vs. model output (y-axis):

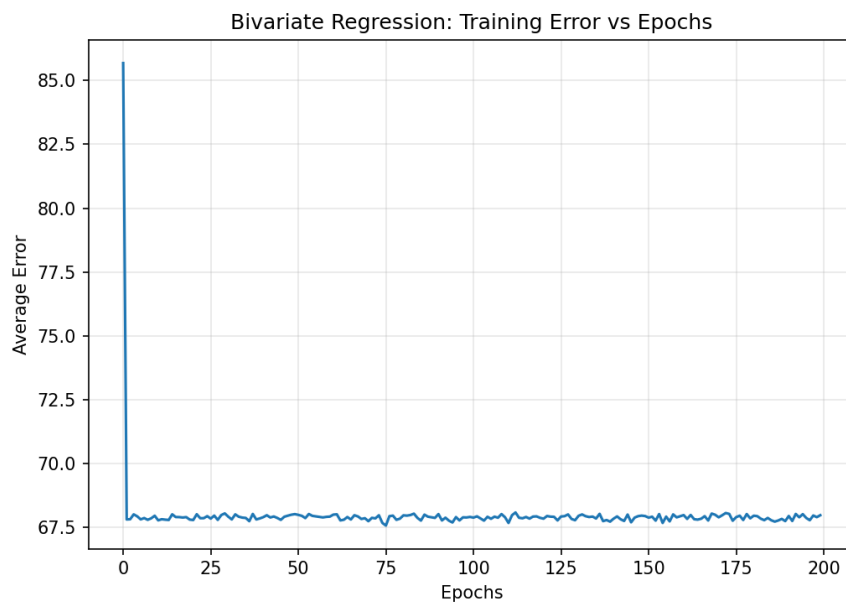


Training data



Test data

4.4 Regression Results — Dataset 2 (Bivariate)

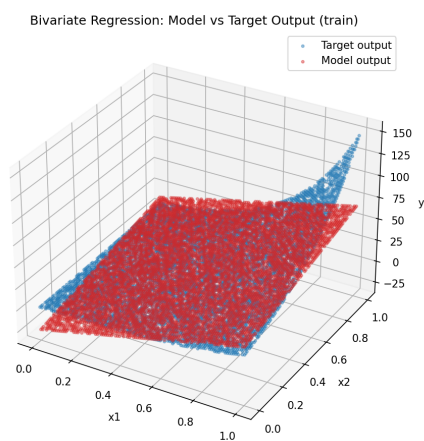


Average error vs. epochs (Dataset 2 (Bivariate)).

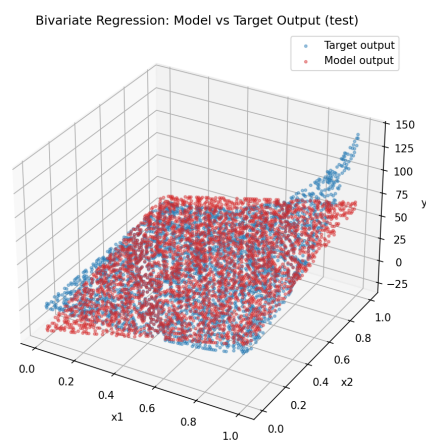
RMSE and %RMSE:

Split	RMSE	%RMSE
Training	11.6178	56.38%
Test	11.5489	56.83%

Model output superimposed on target output (x1, x2 axes: input values, y-axis: target / model output):

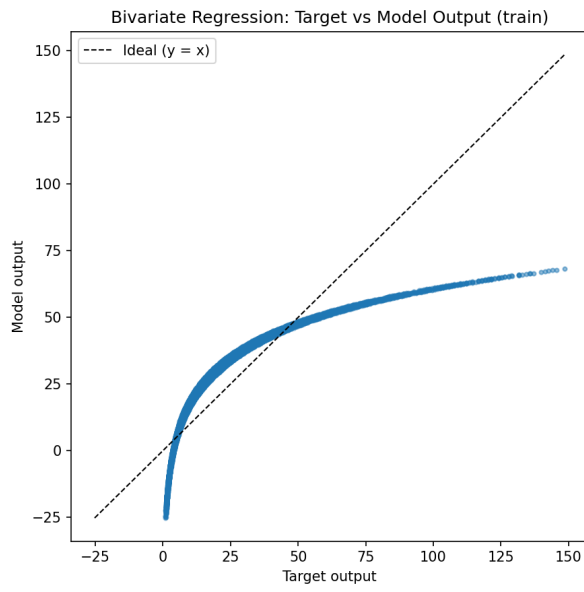


Training data

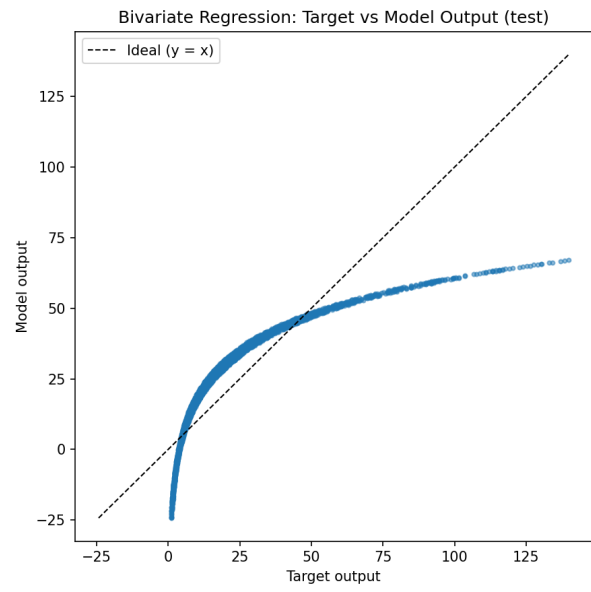


Test data

Scatter plot: target output (x-axis) vs. model output (y-axis):



Training data



Test data

5. Inferences and Observations

5.1 Classification

On the **linearly separable (LS)** dataset, all three one-against-one perceptrons converge to an average error close to zero within a small number of epochs, and the combined decision regions cleanly separate the three classes with straight-line boundaries. This yields 100% test accuracy, 100% precision, recall and F-measure for every class, for both the logistic and tanh activation functions — as expected, since the three classes are, by construction, separable by straight lines and a single perceptron per pair is sufficient to find such a boundary.

On the **non-linearly separable (NLS)** dataset, the three classes form curved, interleaved “moon-shaped” clusters that cannot be separated by straight lines. Because each pairwise perceptron can only learn a linear decision boundary, the combined classifier achieves a lower test accuracy of about 92.9% for both activation functions, with Class 2 (the class sandwiched between the other two) showing the lowest recall and F-measure, since it is most often confused with its neighbours in the pairwise classifiers. The logistic and tanh activations produce almost identical accuracy and confusion patterns; the two activations differ mainly in their output range and in the shape of the error surface during training, but since both ultimately learn the same class of linear decision boundary, their final decision regions and test performance are very similar. This confirms that the performance gap between the LS and NLS datasets is caused by the linear nature of the perceptron's decision boundary rather than by the choice of activation function.

5.2 Regression

In both regression datasets the target output is a visibly non-linear function of the input(s): the univariate target follows a U-shaped (roughly quadratic, symmetric about the middle of the input range) curve, and the bivariate target grows in a curved, super-linear fashion with x_1 and x_2 . Because the perceptron uses a linear activation function, it can only fit a straight line (univariate) or a flat plane (bivariate) to the data — it has no way to represent curvature.

For the univariate case, the best linear fit is nearly flat, since the target is symmetric and rises on both sides of the input range; this gives a relatively high %RMSE of around 42–44% on training and test data. The bivariate case shows a similar pattern with an even higher %RMSE of around 56–57%, reflecting the stronger curvature of that target surface. In both cases the average-error-vs-epochs plot drops sharply within the first few epochs and then plateaus, showing that gradient descent quickly converges to the best possible linear fit — the remaining error is not due to under-training but is an irreducible bias caused by using a linear model for a non-linear relationship. The scatter plots of target vs. model output further confirm this: the points form a roughly horizontal band far from the ideal $y = x$ line, consistent with the model consistently under/over-predicting for large deviations of the input from the middle of its range. Test-set RMSE and %RMSE are close to the corresponding training-set values in both datasets, which shows the perceptron is not overfitting — its limited capacity is simply insufficient to capture the non-linear trend in the data.

6. Conclusion

This assignment implemented the perceptron learning algorithm and its online gradient-descent training rule entirely from scratch, together with custom implementations of every evaluation metric used. Across four experiments — one-against-one classification with logistic and tanh activations on linearly and non-linearly separable data, and linear-activation regression on univariate and bivariate data — a consistent pattern emerges: whenever the true relationship between input and output is linear (or the classes are linearly separable), a single perceptron is entirely sufficient and achieves near-perfect performance. Whenever the true relationship is non-linear, a single perceptron is fundamentally limited by its linear decision boundary / linear output surface, and no amount of additional training epochs or change of activation function can close this gap. This directly motivates the use of multi-layer neural networks with non-linear hidden layers, which are able to combine multiple linear boundaries to approximate arbitrarily non-linear decision regions and regression surfaces.